

4.3

задача

Измеряется *release latency*:

$$L = \max_i(t_i) - t_0,$$

где

- t_0 — момент, когда arbiter выполняет финальный `countDown()`,
- t_i — момент, когда i -й освобождённый поток после выхода из `await()` фиксирует `System.nanoTime()`.

Для каждого значения N создаётся latch со счётчиком $N + 1$.

После этого:

1. запускаются N worker-потоков;
2. каждый worker после общего стартового сигнала выполняет `latch.countDown()`;
3. после этого worker сообщает о прибытии через дополнительный `arrivedGate`;
4. затем worker вызывает `latch.await()`;
5. arbiter ждёт, пока все worker-потоки придут;
6. arbiter фиксирует время t_0 и выполняет финальный `countDown()`;
7. каждый worker после выхода из `await()` фиксирует своё время пробуждения.

Таким образом, измеряется задержка между открытием latch и моментом, когда освободился последний поток.

Реализации

NaiveCountDownLatch

Реализация на одном мониторе:

- `await()` выполняет ожидание в цикле `while (count != 0) wait();`
- финальный `countDown()` делает `notifyAll()`.

MonitorCountDownLatch

Реализация использует отдельный **Token** для каждого ожидающего потока:

- если latch закрыт, поток делает **Token**, добавляет его в очередь ожидающих и ждёт на нём;
- финальный `countDown()` собирает все токены и будит каждый поток отдельным `notify()` на соответствующем токене.

Параметры

- несколько значений числа потоков N ;
- warmup-запуски для прогрева JVM;
- measured-запуски для сбора статистики.

По измерениям вычисляются:

- средняя latency,
- стандартное отклонение.

На графике error bars показывают одно стандартное отклонение.

Результаты

`NaiveCountDownLatch` масштабируется хуже.

Потому что при открытии latch используется `notifyAll()` на одном общем мониторе. После этого все ожидающие потоки просыпаются почти одновременно и начинают конкурировать за один и тот же монитор.

`MonitorCountDownLatch` лучше масштабируется, потому что каждый поток ждёт на собственном объекте **Token**, поэтому финальный поток будит ожидающих по одному, без повторной конкуренции всех потоков за один общий монитор.

График

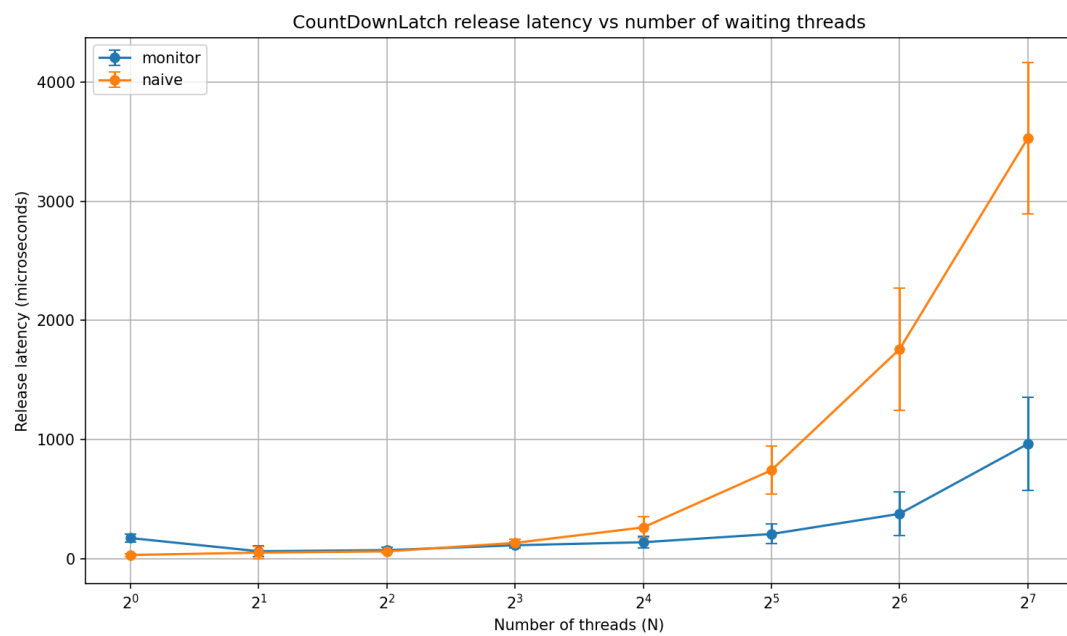


Рис. 1: Release latency for NaiveCountDownLatch and MonitorCountDownLatch